Replacing Web2 Freelance Escrow Intermediaries on Solana

Track: HackYeah - Superteam Poland Track (Replace a Web2 Intermediary)

Target User: Independent Polish/EU B2B Software Engineers & Indie Studios

Redesigned Relationship: Direct Client <-> Contractor Fixed-Scope Sprints

Core Principle: Custody, kill-fees, acceptance timeouts, and dispute splits execute deterministically inside the Solana program with 0% platform cut.

Repository: https://github.com/c-viper-work/clear-milestone-solana

Who Uses ClearMilestone & Why Web2 Fails Them

Specific User: Independent Polish B2B software/game engineer (Kontraktor B2B)

delivering a 1-to-4 week milestone for an overseas studio or startup client.

Pain Point 1 (Fee Extraction): Web2 marketplaces (Upwork/Fiverr/Escrow.com)

take 10%-20% of contract value plus 2%-4% cross-border FX conversion spreads.

Pain Point 2 (Payout Delay): Even after client approval, funds are held 5-14 days.

Pain Point 3 (Asymmetric Risk): Clients can cancel mid-sprint with $0 kill-fee

or ghost after receiving code, freezing contractor payout behind support tickets.

What Changes When the Intermediary Becomes a Solana Program

Before (Web2): Client -> Platform Custodian -> Opaque Ticket Queue -> Contractor

After (Solana): Client <-> SprintEscrow PDA <-> Contractor (+ Optional Arbiter)

1. Upfront Principal Lock: Client locks exact lamports in a deterministic PDA.
2. Cryptographic Delivery Proof: Contractor commits SHA-256 of deliverable bytes on-chain, starting the authoritative Solana Clock review countdown.
3. Zero Platform Rent-Seeking: 0% protocol fee; 100% of principal is conserved across contractor, client, and agreed arbiter basis points.

All Intermediary Logic Lives in programs/clear-milestone/src/lib.rs

Deterministic Seeds: [b"sprint", client, contractor, sprint_id.to_le_bytes()]

Fixed 386-Byte Account Layout: 3 distinct party Pubkeys, sprint_id, principal,

kill_fee_bps (<=5000), arbiter_fee_bps (<=500), review_window_secs (<=30d),

created_at, submitted_at, auto_release_at, spec/deliverable/dispute SHA-256,

bounded UTF-8 deliverable URI (<=128B), status enum (0..=6), canonical bump.

Strict 4-Address Distinctness: Escrow PDA, Client, Contractor, and Arbiter

must all be distinct public keys.

Complete Lifecycle Without Backend Servers

1. initialize_sprint: Validates terms & transfers principal into SprintEscrow PDA.
2. submit_deliverable: Contractor records SHA-256 + URI; sets auto_release_at.
3. approve_deliverable: Client releases 100% of principal to Contractor.
4. claim_after_timeout: If Clock.unix_timestamp >= auto_release_at in Submitted state, Contractor claims 100% of principal without waiting for a ghosting client.
5. cancel_before_submission: Client cancels in Funded state; program atomically pays kill_fee_bps to Contractor and refunds the exact remainder to Client.

6-7. raise_dispute & resolve_dispute: Freezes timeout; Arbiter splits remainder.

Engineered for Real-World Solana Runtime Constraints

1. Exact Lamport Conservation: Every settlement path (Approve, Claim, Cancel, Resolve) is proven across all basis points (0..=10000) and up to u64::MAX.
2. Escrow Rent Preservation: settle() verifies PDA retains its own rent-exempt minimum plus full principal before executing any debit.
3. Recipient Rent-Exemption Guard (RecipientBelowRent): Prevents Solana SVM InsufficientFundsForRent aborts when paying small splits to 0-lamport accounts; the UI automatically prepends atomic SystemProgram.transfer top-ups.

Connect Wallet -> Trigger Transaction -> Inspect on Solana Explorer

1. Wallet & Judge Modes: Supports Phantom, Solflare, and Backpack extensions, plus one-click in-memory 3-role Ephemeral Judge Signers (Client/Contractor/Arbiter).
2. Local WebCrypto Hashing: Calculates SHA-256 of sprint specs, deliverable files, and dispute evidence directly in the browser (no file upload or backend).
3. Fail-Closed RPC & Broadcast Safety: Verifies Devnet genesis hash before signing, extracts local Base58 signatures prior to broadcast, and links every tx and PDA directly to Solana Explorer (Devnet).

751 Frontend/Wire Checks + 11 Rust Invariant Tests + SBF Compilation

- Rust State-Machine & Boundary Verifier: 11/11 tests passing (all 8 statuses x 4 roles x 6 actions x 3 deadline states, plus rent & 4-address boundaries).
- Independent Wire & Borsh Oracle (scripts/check-frontend.py): 751/751 checks passing against compiled Rust oracle and independent SHA-256/PDA vectors.
- Solana SBF Release Build: Compiled cleanly to optimized SBF ELF (.so) and Anchor JSON IDL via scripts/playground-build.py.

Structural Comparison

Metric	Web2 Escrow (Upwork/Escrow)	ClearMilestone (Solana)
Platform / Escrow Fee	10% - 20% + FX spread	0% protocol fee
Payout Clearing Delay	5 - 14 business days	Immediate finality
Client Ghosting Remedy	Manual support ticket	Clock timeout claim
Early Cancel Kill-Fee	Discretionary / uncollectible	Atomic kill_fee split
Deliverable Timestamp	Mutable platform database	SHA-256 + Solana Clock

Open Source & Reproducible in Seconds

```
Public GitHub Repo: https://github.com/c-viper-work/clear-milestone-solana

Live Web Console:   https://c-viper-work.github.io/clear-milestone-solana/app/

Run Rust Verifier:  cargo test --offline --manifest-path

                    crates/state-machine-verifier/Cargo.toml

Run 751 Wire Tests: python3 scripts/check-frontend.py

Serve Console:      python3 -m http.server 8080 --bind 127.0.0.1 --directory app
```